

# QuantMath Library

Development Journal

Ben Fielding

September 17, 2026

# Contents

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                     | <b>3</b>  |
| <b>2</b> | <b>Library Design</b>                   | <b>4</b>  |
| <b>3</b> | <b>Vector Class</b>                     | <b>5</b>  |
| 3.1      | Purpose . . . . .                       | 5         |
| 3.2      | Construction . . . . .                  | 5         |
| 3.3      | Vector Size . . . . .                   | 6         |
| 3.4      | Element Access . . . . .                | 6         |
| 3.5      | Negative Indexing . . . . .             | 7         |
| 3.6      | Bounds Checking . . . . .               | 7         |
| 3.7      | Vector Addition . . . . .               | 8         |
| 3.8      | Vector Subtraction . . . . .            | 8         |
| 3.9      | Scalar Multiplication . . . . .         | 9         |
| 3.10     | Scalar Division . . . . .               | 9         |
| 3.11     | Unary Negation . . . . .                | 9         |
| 3.12     | Compound Assignment . . . . .           | 10        |
| 3.13     | Equality and Inequality . . . . .       | 10        |
| 3.14     | Vector Norm . . . . .                   | 11        |
| 3.15     | Dot Product . . . . .                   | 11        |
| 3.16     | Normalisation . . . . .                 | 12        |
| 3.17     | Distance Between Vectors . . . . .      | 12        |
| 3.18     | Angle Between Vectors . . . . .         | 12        |
| 3.19     | Orthogonality . . . . .                 | 13        |
| 3.20     | Projection . . . . .                    | 13        |
| 3.21     | Cross Product . . . . .                 | 13        |
| 3.22     | Stream Output . . . . .                 | 14        |
| 3.23     | Lessons From the Vector Class . . . . . | 14        |
| <b>4</b> | <b>Matrix Class</b>                     | <b>15</b> |
| 4.1      | Purpose . . . . .                       | 15        |
| 4.2      | Matrix Storage . . . . .                | 15        |
| 4.3      | Rows and Columns . . . . .              | 16        |
| 4.4      | Element Access . . . . .                | 17        |
| 4.5      | Negative Matrix Indexing . . . . .      | 17        |
| 4.6      | Bounds Checking . . . . .               | 18        |
| 4.7      | Matrix Output . . . . .                 | 18        |
| 4.8      | Matrix Addition . . . . .               | 18        |
| 4.9      | Matrix Subtraction . . . . .            | 19        |
| 4.10     | Scalar Multiplication . . . . .         | 19        |
| 4.11     | Scalar Division . . . . .               | 20        |
| 4.12     | Unary Negation . . . . .                | 20        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 4.13     | Compound Matrix Operations . . . . .          | 20        |
| <b>5</b> | <b>Current Project Status</b>                 | <b>22</b> |
| 5.1      | Vector . . . . .                              | 22        |
| 5.2      | Matrix . . . . .                              | 23        |
| <b>6</b> | <b>Numerical Methods Development Process</b>  | <b>24</b> |
| <b>7</b> | <b>Future Development</b>                     | <b>26</b> |
| 7.1      | Equality and Inequality . . . . .             | 26        |
| 7.2      | Matrix–Vector Multiplication . . . . .        | 27        |
| 7.3      | Column Extraction . . . . .                   | 27        |
| 7.4      | Zero Matrices . . . . .                       | 28        |
| 7.5      | Matrix–Matrix Multiplication . . . . .        | 28        |
| 7.6      | Matrix Transpose . . . . .                    | 29        |
| 7.7      | Final Matrix Testing . . . . .                | 30        |
| 7.8      | Completion of the Core Matrix Class . . . . . | 31        |

# Chapter 1

## Introduction

The **QuantMath Library** is a personal C++ mathematical and numerical-computing library developed from first principles.

The purpose of the project is not simply to reproduce functionality already available in libraries such as NumPy, Eigen, or SciPy. Instead, the project is being used to develop a deeper understanding of

- object-oriented programming in C++,
- mathematical data structures,
- operator overloading,
- numerical algorithms,
- linear algebra,
- numerical analysis,
- software design,
- testing,
- and mathematical implementation.

The project begins with the fundamental mathematical objects required by later numerical algorithms.

At the current stage of development, the library contains completed core Vector and Matrix classes. Development will now move into numerical methods, with more advanced linear algebra functionality introduced later when required.

Later stages will translate numerical methods previously explored in Python into a structured C++ library.

## Chapter 2

# Library Design

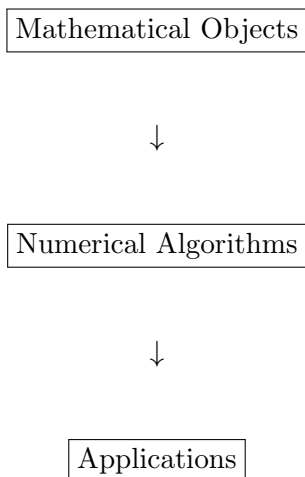
A major goal of QuantMath is to keep the mathematical objects themselves separate from the numerical algorithms which will eventually operate on them.

For example,

- a **Vector** object should represent and manipulate a mathematical vector,
- a **Matrix** object should represent and manipulate a mathematical matrix,
- root-finding algorithms should later exist as numerical algorithms which use appropriate mathematical objects,
- differential equation solvers should be separate from the fundamental storage classes.

This separation prevents the classes from becoming unnecessarily large and creates a more modular mathematical library.

The general architecture is therefore



The first two mathematical objects developed are vectors and matrices.

## Chapter 3

# Vector Class

### 3.1 Purpose

The first major component developed for QuantMath was the **Vector** class.

Mathematically, an  $n$ -dimensional real vector may be written as

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}.$$

In C++, the components are stored dynamically using

```
1 std::vector<double>
```

rather than using a fixed-size array.

This allows the class to represent vectors of arbitrary finite dimension.

Conceptually, the class stores

$$\mathbf{v} = (v_0, v_1, \dots, v_{n-1}).$$

Although mathematical notation normally begins at index 1, C++ containers begin at index 0.

### 3.2 Construction

The class was designed to allow vectors to be created naturally from a collection of values.

For example, a three-dimensional vector may conceptually be constructed as

```
1 Vector v{1.0, 2.0, 3.0};
```

representing

$$\mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

Utility constructors or factory-style operations were also developed for commonly required vectors such as

$$\mathbf{0} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix},$$

and vectors containing repeated values.

These operations provide convenient ways to create vectors without manually specifying every component.

### 3.3 Vector Size

A vector must know its dimension.

If

$$\mathbf{v} \in \mathbb{R}^n,$$

then its size is  $n$ .

The class therefore provides access to the number of stored components. This becomes especially important when performing arithmetic because two vectors must normally have compatible dimensions.

For example,

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$$

is not defined.

Dimension checking is therefore an important part of the class design.

### 3.4 Element Access

One of the important features implemented in the Vector class was element access using the square-bracket operator.

This allows syntax of the form

```
1 v[0]
```

to access a component.

Both writable and constant versions of the operator are useful.

The writable version allows

```
1 v[0] = 5.0;
```

while the constant version allows components of a constant vector to be read safely without modifying the object.

### 3.5 Negative Indexing

A deliberate design feature of the QuantMath Vector class was support for Python-style negative indexing.

For a vector

$$\mathbf{v} = (v_0, v_1, \dots, v_{n-1}),$$

the index

$$-1$$

refers to the final element,

$$-2$$

refers to the second-to-last element, and so on.

The conversion is based on

$$i_{\text{converted}} = n + i, \quad i < 0.$$

Therefore, for a vector of size 5,

$$-1 \mapsto 4,$$

$$-2 \mapsto 3,$$

and

$$-5 \mapsto 0.$$

This allows syntax such as

```
1 v[-1]
```

to return the final component.

This feature was included because it provides a convenient interface similar to Python while still being implemented explicitly in C++.

### 3.6 Bounds Checking

Negative indexing introduces an important problem: not every negative index is valid.

For a vector of size  $n$ , the acceptable indices are

$$-n \leq i < n.$$

Indices outside this range should not be allowed to access memory.

The class therefore performs bounds checking before returning a component.

Invalid indices generate an exception rather than silently producing undefined behaviour.

This is particularly important in a mathematical library, where an incorrect memory access could otherwise produce numerical results which appear valid but are actually meaningless.

### 3.7 Vector Addition

Vector addition was implemented using operator overloading.

For

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix},$$

addition is defined componentwise:

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{bmatrix}.$$

The C++ interface therefore allows natural syntax such as

```
1 Vector w = u + v;
```

rather than requiring a function call such as

```
1 Vector w = add(u, v);
```

This makes the program resemble the corresponding mathematics.

### 3.8 Vector Subtraction

Subtraction is implemented similarly.

Mathematically,

$$\mathbf{u} - \mathbf{v} = \begin{bmatrix} u_1 - v_1 \\ u_2 - v_2 \\ \vdots \\ u_n - v_n \end{bmatrix}.$$

The corresponding C++ expression is

```
1 Vector w = u - v;
```

Again, the dimensions of the vectors must be compatible.

### 3.9 Scalar Multiplication

For a scalar

$$\lambda \in \mathbb{R},$$

and vector

$$\mathbf{v} = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix},$$

scalar multiplication is

$$\lambda \mathbf{v} = \begin{bmatrix} \lambda v_1 \\ \vdots \\ \lambda v_n \end{bmatrix}.$$

Operator overloading allows this to be expressed naturally in C++.

The class supports multiplication of vector components by a scalar, and the library design also allows scalar–vector syntax to behave in a mathematically intuitive way.

### 3.10 Scalar Division

Division by a scalar was also implemented.

For

$$\lambda \neq 0,$$

$$\frac{\mathbf{v}}{\lambda} = \begin{bmatrix} v_1/\lambda \\ \vdots \\ v_n/\lambda \end{bmatrix}.$$

This operation requires consideration of division by zero, since the operation

$$\frac{\mathbf{v}}{0}$$

is undefined.

### 3.11 Unary Negation

The unary minus operator represents multiplication by  $-1$ .

Therefore,

$$-\mathbf{v} = \begin{bmatrix} -v_1 \\ -v_2 \\ \vdots \\ -v_n \end{bmatrix}.$$

This allows expressions such as

```
1 Vector u = -v;
```

and helps make later mathematical expressions much cleaner.

### 3.12 Compound Assignment

Compound assignment operators were introduced to modify an existing vector directly.

For example,

```
1 u += v;
```

is mathematically equivalent to

$$\mathbf{u} \leftarrow \mathbf{u} + \mathbf{v}.$$

Similarly,

```
1 u -= v;
```

corresponds to

$$\mathbf{u} \leftarrow \mathbf{u} - \mathbf{v}.$$

These operations also introduced the use of

```
1 *this
```

inside member functions.

The expression `*this` represents the current object and allows an overloaded operator to return a reference to the modified vector.

### 3.13 Equality and Inequality

Equality operators were introduced so that two vectors can be compared.

Rather than requiring code such as

```
1 compare_vectors(u, v)
```

the interface allows

```
1 u == v
```

and

```
1 u != v
```

to be used directly.

For floating-point calculations, exact equality can be problematic because computer representations of real numbers are finite.

For example, calculations which are mathematically identical may produce slightly different floating-point values.

For this reason, numerical comparison can use a small tolerance

$$\varepsilon > 0$$

and regard two components as equal when

$$|u_i - v_i| < \varepsilon.$$

This is a recurring idea in numerical computing and will become increasingly important throughout the QuantMath project.

### 3.14 Vector Norm

The Euclidean norm of a vector was introduced as a fundamental vector operation.

For

$$\mathbf{v} = (v_1, v_2, \dots, v_n),$$

the Euclidean norm is

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}.$$

Equivalently,

$$\|\mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n v_i^2}.$$

This provides a measure of the length or magnitude of the vector.

### 3.15 Dot Product

For two vectors

$$\mathbf{u}, \mathbf{v} \in \mathbb{R}^n,$$

the dot product is

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i.$$

For example,

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 1(4) + 2(5) + 3(6) = 32.$$

The dot product returns a scalar rather than another vector.

It is fundamental to many later topics including

- orthogonality,

- projections,
- matrix multiplication,
- least squares,
- optimisation,
- and numerical linear algebra.

### 3.16 Normalisation

A non-zero vector can be converted into a unit vector by dividing by its norm.

For

$$\mathbf{v} \neq \mathbf{0},$$

define

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}.$$

Then

$$\|\hat{\mathbf{v}}\| = 1.$$

This operation was incorporated into the Vector class as a natural extension of scalar division and the norm operation.

### 3.17 Distance Between Vectors

Vectors can also represent points in Euclidean space.

The distance between two vectors is therefore

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|.$$

This reuses functionality already implemented in the class:

1. subtract the vectors,
2. calculate the norm of the result.

This demonstrates an important software-design principle: new mathematical operations should reuse existing functionality where possible rather than reimplementing the same calculations.

### 3.18 Angle Between Vectors

The dot-product identity

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

can be rearranged to give

$$\theta = \cos^{-1} \left( \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \right).$$

This provides the angle between two non-zero vectors.

The implementation therefore combines existing operations rather than introducing an entirely separate calculation.

### 3.19 Orthogonality

Two vectors are orthogonal when

$$\mathbf{u} \cdot \mathbf{v} = 0.$$

Because floating-point arithmetic is approximate, numerical orthogonality is better tested using a tolerance.

Thus, in practice,

$$|\mathbf{u} \cdot \mathbf{v}| < \varepsilon$$

is used rather than demanding exact zero.

### 3.20 Projection

The scalar projection of  $\mathbf{u}$  onto  $\mathbf{v}$  is

$$\text{comp}_{\mathbf{v}} \mathbf{u} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|}.$$

The vector projection is

$$\text{proj}_{\mathbf{v}} \mathbf{u} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|^2} \mathbf{v}.$$

These operations are constructed using the already implemented dot product, norm, and scalar multiplication functionality.

### 3.21 Cross Product

For three-dimensional vectors,

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix},$$

their cross product is

$$\mathbf{u} \times \mathbf{v} = \begin{bmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{bmatrix}.$$

Unlike the dot product, the cross product produces another vector.

The implementation therefore needs to enforce the dimensional requirement that both inputs belong to

$$\mathbb{R}^3.$$

## 3.22 Stream Output

The stream insertion operator was overloaded so that vectors can be printed directly.

Instead of requiring a dedicated call such as

```
1 v.display();
```

the standard C++ interface can be used:

```
1 std::cout << v;
```

This is a more natural interface and allows Vector objects to behave similarly to built-in C++ types.

The operator is implemented outside the class because the left-hand operand is an output stream rather than a Vector object.

## 3.23 Lessons From the Vector Class

The Vector class introduced several important C++ ideas which will be reused throughout QuantMath:

- classes and encapsulation,
- header and source separation,
- constructors,
- references,
- const correctness,
- operator overloading,
- exceptions,
- dynamic storage,
- reusable mathematical methods,
- floating-point tolerances,
- friend or non-member operators,
- and returning references to `*this`.

The Vector class therefore acts as the foundation for the Matrix class and later numerical algorithms.

## Chapter 4

# Matrix Class

### 4.1 Purpose

After developing the Vector class, the next stage of QuantMath was the construction of a Matrix class.

A matrix with  $m$  rows and  $n$  columns has the form

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

We write

$$A \in \mathbb{R}^{m \times n}.$$

The Matrix class is designed to become the foundation for later topics such as

- systems of linear equations,
- Gaussian elimination,
- matrix decompositions,
- regression,
- numerical differential equations,
- optimisation,
- and numerical linear algebra.

### 4.2 Matrix Storage

Rather than storing all matrix entries in a single unstructured container, the Matrix class was designed using the existing Vector class.

Conceptually, the storage is

```
1 std::vector<Vector> data;
```

where each Vector represents one row of the matrix.

For example,

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

can be viewed internally as two row vectors,

$$\mathbf{r}_1 = (1, 2, 3),$$

and

$$\mathbf{r}_2 = (4, 5, 6).$$

This design allows the Matrix class to build directly upon the Vector class rather than creating an entirely separate storage system.

This is one of the first examples in QuantMath of composing a more complex mathematical object from a simpler object which has already been implemented.

### 4.3 Rows and Columns

A matrix must track two dimensions.

For

$$A \in \mathbb{R}^{m \times n},$$

$m$  represents the number of rows and  $n$  represents the number of columns.

Functions were therefore introduced to access these dimensions.

These values are required when validating arithmetic operations.

For example, matrices

$$A \in \mathbb{R}^{m \times n}$$

and

$$B \in \mathbb{R}^{p \times q}$$

can only be added when

$$m = p$$

and

$$n = q.$$

## 4.4 Element Access

Unlike Vector, where a single index is sufficient, a matrix entry requires both a row and a column.

QuantMath therefore uses function-call syntax

```
1 A(i, j)
```

to access the entry in row  $i$  and column  $j$ .

This is achieved by overloading

```
1 operator()
```

rather than using nested square brackets.

The notation

```
1 A(i, j)
```

also resembles standard mathematical notation

$$a_{ij}.$$

Both writable and constant forms are required.

The writable form allows

```
1 A(0, 1) = 5.0;
```

while the const form allows safe access to entries of a constant Matrix.

## 4.5 Negative Matrix Indexing

The Python-style negative indexing convention introduced for Vector was also extended to Matrix.

For a matrix with  $m$  rows and  $n$  columns,

```
1 A(-1, -1)
```

refers to the entry in the final row and final column.

Negative row indices are converted using

$$i_{\text{converted}} = m + i,$$

while negative column indices use

$$j_{\text{converted}} = n + j.$$

This produces a consistent interface between Vector and Matrix.

## 4.6 Bounds Checking

Matrix element access also requires bounds checking.

For a matrix

$$A \in \mathbb{R}^{m \times n},$$

a valid row index must satisfy

$$-m \leq i < m,$$

and a valid column index must satisfy

$$-n \leq j < n.$$

Any index outside the permitted range should generate an exception.

This protects the program from invalid memory access and ensures that indexing errors are detected explicitly.

## 4.7 Matrix Output

The stream insertion operator was extended to Matrix objects.

This allows

```
1 std::cout << A;
```

to produce a readable matrix representation.

For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

can be displayed row-by-row.

As with Vector, overloading the stream operator makes the class easier to test and more natural to use.

## 4.8 Matrix Addition

For two matrices

$$A, B \in \mathbb{R}^{m \times n},$$

addition is defined elementwise:

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

For example,

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}.$$

The Matrix class therefore overloads the addition operator to allow

```
1 Matrix C = A + B;
```

provided that the two matrices have identical dimensions.

## 4.9 Matrix Subtraction

Subtraction is similarly performed elementwise.

$$(A - B)_{ij} = a_{ij} - b_{ij}.$$

Therefore,

```
1 Matrix C = A - B;
```

corresponds directly to the mathematical operation

$$C = A - B.$$

Again, matrix dimensions must agree.

## 4.10 Scalar Multiplication

For

$$A \in \mathbb{R}^{m \times n}$$

and

$$\lambda \in \mathbb{R},$$

scalar multiplication is defined as

$$(\lambda A)_{ij} = \lambda a_{ij}.$$

For example,

$$3 \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}.$$

The Matrix class therefore supports multiplication of every entry by a scalar.

## 4.11 Scalar Division

Similarly,

$$\frac{A}{\lambda}$$

is obtained by dividing each entry by  $\lambda$ , provided

$$\lambda \neq 0.$$

Thus,

$$\left(\frac{A}{\lambda}\right)_{ij} = \frac{a_{ij}}{\lambda}.$$

As with Vector, division by zero must be handled explicitly.

## 4.12 Unary Negation

Matrix negation is defined by

$$(-A)_{ij} = -a_{ij}.$$

Therefore,

$$-A = (-1)A.$$

This allows C++ expressions such as

```
1 Matrix B = -A;
```

to match standard mathematical notation.

## 4.13 Compound Matrix Operations

Compound assignment operators were also introduced.

For example,

```
1 A += B;
```

represents

$$A \leftarrow A + B,$$

while

```
1 A -= B;
```

represents

$$A \leftarrow A - B.$$

These operations reuse the ideas developed while implementing the Vector class.

This demonstrates one of the advantages of developing the library incrementally: concepts learned while implementing one mathematical object can be reused when constructing the next.

## Chapter 5

# Current Project Status

At the present stage, the two fundamental data structures being developed are Vector and Matrix.

### 5.1 Vector

The Vector work has covered the central interface and mathematical operations required for the library, including

- dynamic component storage,
- construction,
- size and component access,
- const and non-const indexing,
- Python-style negative indexing,
- bounds checking,
- addition and subtraction,
- scalar multiplication and division,
- unary negation,
- compound assignment,
- equality and inequality,
- stream output,
- norms,
- dot products,
- normalisation,
- distances,
- angles,
- projections,
- orthogonality,
- and three-dimensional cross products.

The Vector class now provides the mathematical foundation required by the Matrix class.

## 5.2 Matrix

The core Matrix phase is now complete.

The class currently supports

- matrix storage and construction,
- row and column dimensions,
- element access and negative indexing,
- bounds checking,
- arithmetic operators,
- equality and inequality,
- matrix–vector multiplication,
- matrix–matrix multiplication,
- column extraction,
- zero-matrix construction,
- transpose,
- and exception handling for invalid operations.

Advanced linear algebra operations will be introduced later as part of a dedicated numerical linear algebra phase.

## Chapter 6

# Numerical Methods Development Process

Once the core Matrix implementation is complete, the next major stage of QuantMath will involve translating numerical methods previously studied in Python into C++.

However, these algorithms will not simply be copied from Python.

Each method will instead be developed from first principles using the following process:

Mathematics on Paper



Derivation



Pseudocode



C++ Design



Implementation



Testing

↓

LaTeX Journal

↓

Git Commit / Push

The purpose of this process is to ensure that every algorithm in the library is mathematically understood before it is implemented.

The LaTeX journal will therefore become both

1. a record of the development of the QuantMath library, and
2. a mathematical reference explaining the algorithms contained within it.

# Chapter 7

## Future Development

Once the Matrix class is sufficiently complete, development will move into numerical methods.

The current intended progression is

Vector  $\rightarrow$  Matrix  $\rightarrow$  Root Finding  $\rightarrow$  Integration  $\rightarrow$  Differentiation  $\rightarrow$  Interpolation

followed by

Linear Algebra  $\rightarrow$  ODE Solvers  $\rightarrow$  Optimisation  $\rightarrow$  Simulation and Advanced Numerical Methods.

Each new section of the library will be added as a new chapter of this journal.

Consequently, this document is intended to develop alongside the software rather than being written only after the project is complete.

“latex

### 7.1 Equality and Inequality

Equality and inequality operators were implemented for matrices in the same way as for vectors.

Two matrices can only be equal if they have the same dimensions. Thus, for

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{p \times q},$$

equality first requires

$$m = p \quad \text{and} \quad n = q.$$

If the dimensions agree, the corresponding row vectors are compared.

This allows expressions such as

```
1 A == B
```

and

```
1 A != B
```

to be used directly.

The implementation reuses the equality operation already developed for the Vector class, demonstrating how the Matrix class can build upon existing functionality rather than reproducing it.

## 7.2 Matrix–Vector Multiplication

The Matrix class was extended to support multiplication by a Vector.

Let

$$A \in \mathbb{R}^{m \times n}$$

and

$$\mathbf{x} \in \mathbb{R}^n.$$

Then

$$A\mathbf{x} \in \mathbb{R}^m.$$

Each component of the result is obtained by taking the dot product of one row of the matrix with the vector:

$$(A\mathbf{x})_i = \sum_{j=1}^n a_{ij}x_j.$$

Equivalently,

$$(A\mathbf{x})_i = \text{row}_i(A) \cdot \mathbf{x}.$$

This allowed the existing Vector dot-product method to be reused.

For example,

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 14 \\ 32 \end{bmatrix}.$$

Matrix–vector multiplication is only defined when the number of columns of the matrix equals the dimension of the vector.

## 7.3 Column Extraction

Because the internal Matrix representation stores rows as Vector objects, accessing a row is straightforward.

Matrix multiplication, however, also requires access to columns.

A `column()` method was therefore implemented which constructs and returns a Vector containing a specified matrix column.

For

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix},$$

the first column is

$$\begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix}.$$

The column index is checked before extraction, and an invalid column generates an exception.

## 7.4 Zero Matrices

A general matrix constructor was introduced which allows a matrix of specified dimensions to be filled with a chosen value.

This made it possible to implement a convenient zero-matrix factory.

For

$$m, n \in \mathbb{N},$$

the matrix

$$0_{m \times n}$$

contains  $m$  rows,  $n$  columns, and every entry is zero.

For example,

$$0_{3 \times 4} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

In C++, a zero matrix can be created using

```
1 Matrix Z = Matrix::zeros(3, 4);
```

This became particularly useful when implementing matrix multiplication, where the dimensions of the result are not necessarily the same as either input matrix.

## 7.5 Matrix–Matrix Multiplication

Matrix multiplication was the most substantial operation added during the core Matrix phase.

Let

$$A \in \mathbb{R}^{m \times n}$$

and

$$B \in \mathbb{R}^{n \times p}.$$

The product

$$C = AB$$

has dimensions

$$C \in \mathbb{R}^{m \times p}.$$

The multiplication is only defined when the number of columns of the first matrix equals the number of rows of the second:

$$\boxed{\text{columns}(A) = \text{rows}(B)}$$

Each entry of the product is

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

This can also be interpreted as the dot product

$$\boxed{c_{ij} = \text{row}_i(A) \cdot \text{column}_j(B)}$$

which fits naturally with the existing Vector implementation.

The result matrix is initially created using

```
1 Matrix::zeros(rows(), other.columns())
```

and each entry is then replaced by the appropriate row-column dot product.

This design allowed matrix multiplication to reuse both the Vector dot-product method and the Matrix column-extraction method.

## 7.6 Matrix Transpose

The transpose operation was implemented using the method

```
1 A.T()
```

For a matrix

$$A \in \mathbb{R}^{m \times n},$$

its transpose satisfies

$$A^T \in \mathbb{R}^{n \times m}.$$

The entries obey

$$(A^T)_{ji} = A_{ij}.$$

For example,

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

has transpose

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

The implementation therefore creates a zero matrix with dimensions

$$n \times m$$

and places each entry  $A_{ij}$  into position  $(j, i)$  of the result.

## 7.7 Final Matrix Testing

Before completing the core Matrix phase, several tests were performed to verify both valid calculations and invalid operations.

A rectangular matrix multiplication was tested using

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}.$$

The implementation produced

$$AB = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix},$$

confirming that matrix multiplication correctly handles non-square matrices.

Matrix–vector multiplication produced

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 14 \\ 32 \end{bmatrix}.$$

The transpose test correctly produced

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Negative indexing was also tested successfully, including access to the final matrix entry using

```
1 A(-1, -1)
```

Invalid operations were deliberately tested to ensure that the class fails safely.

These included

- incompatible matrix multiplication,

- out-of-range matrix indexing,
- scalar division by zero,
- and invalid column extraction.

Each generated the expected C++ exception rather than producing an invalid numerical result.

## 7.8 Completion of the Core Matrix Class

The completion of matrix multiplication, zero-matrix construction, transpose, and final testing marks the end of the initial Matrix phase of QuantMath.

The core Matrix class now provides

- dynamic matrix storage using Vector rows,
- construction and dimension access,
- const and non-const element access,
- Python-style negative indexing,
- bounds checking,
- matrix addition and subtraction,
- scalar multiplication and division,
- unary negation,
- compound arithmetic operators,
- equality and inequality,
- matrix–vector multiplication,
- column extraction,
- zero-matrix construction,
- matrix–matrix multiplication,
- transpose,
- and exception handling for invalid operations.

More advanced operations such as determinants, matrix inversion, Gaussian elimination, matrix decompositions, eigenvalues, and eigenvectors are intentionally postponed.

These operations will be developed later as part of the numerical linear algebra phase, where the underlying mathematics and numerical algorithms can be studied before implementation. ““